

PagedAttention

解决问题：传统的 KV Cache 依赖最大请求长度来预分配内存

导致了显存空间的大量浪费 空间利用率低下

机制逻辑：

1. 内存切割：把 KV Cache 分割成较小的页
2. 逻辑映射：类似虚拟内存管理 将连续的逻辑空间映射到离散的物理空间上
3. 按需分配：按照需求分配 不进行预分配

效果：极大提高了显存的利用率(40%→90%) 也能在同时处理更多请求和数据

(PagedAttention 优化了资源的管理策略，并不改变模型计算逻辑)

RadixAttention

解决问题:多轮请求中，相同前缀的重复计算导致算力资源浪费

机制逻辑：

0.前提：相同 token 导出的结果不随环境改变(确定性)

- 1.基数树：用树存储 KV Cache，将没有分支的链压缩成一个节点
- 2.匹配最长前缀：先遍历基数树，寻找最长匹配分支
- 3.缓存命中：匹配的部分复用 KV Cache，仅计算新长出来的部分
(此处进行分支，成为树的一部分)
4. 内存回收：释放最久没调用的叶子

效果：官方数据显示吞吐量极大提高(4.5 倍)